GENERALITAT VALENCIANA
Conselleria d'Educació, Cultura i Esport

IES JUAN DE GARAY
C/ Juan de Garay, 25 46017 Valencia
Tel: 961 20 60 45 Fax: 961 20 60 46
46012963@gva.es www.juandegaray.es

GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN Y FORMACIÓN PROFESIONAL

Unión Europea
Fondo Social Europeo
El FSE invierte en tu futuro

UD10
BOLETÍN EJERCICIOS VARIOS

Bases de Datos
1° DAW/DAM

Abelardo Martínez

Elena González Moreno

Versión:260427.1034

Licencia



Reconocimiento – NoComercial – CompartirIgual (by-nc-sa): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:

⚠ Importante

⚛ Interesante

ÍNDICE DE CONTENIDO

1. BASE DE DATOS A UTILIZAR....4
Cuidado se parece a UD05_jardineria pero no es igual....4
2. Ejercicio 1: Trigger para actualizar stock....4
2.1 Para probar el Trigger....4
3. Trigger para registrar cambios de estado en pedidos....4
3.1 Para probar el Trigger....5
4. Trigger para evitar pedidos con cantidades negativas....5
4.1 Para probar el Trigger....5
5. Cursor para mostrar clientes de una ciudad....6
5.1 Cómo probar el cursor....6
6. Ejercicio 5: Cursor para calcular el total de compras de cada cliente....6
6.1 Cómo probar el cursor....6
7. Ejercicio 6: Cursor para listar productos con bajo stock....7
7.1 Cómo probar el cursor....7
8. Ejercicio 7: Cursor para contar empleados por oficina....7
8.1 Cómo probar el cursor....7
9. Ejercicio 8: Cursor para contar empleados por CIUDAD....8
9.1 Cómo probar el cursor....8
10. Ejercicio 9: Función para calcular el total de un pedido....8
10.1 Cómo probar la función....8

UD010. BOLETÍN EJERCICIOS VARIOS

1. BASE DE DATOS A UTILIZAR

Cuidado se parece a UD05_jardineria pero no es igual.

La puedes encontrar en Recursos Complementarios del tema.

2. EJERCICIO 1: TRIGGER PARA ACTUALIZAR STOCK

Crear un trigger que, al insertar una nueva línea en la tabla `detalle_pedido`, reste la cantidad pedida del stock del producto correspondiente en la tabla `productos`.

2.1 Para probar el Trigger

-- Compruebas cuantos hay en stock

SELECT CantidadEnStock FROM Productos WHERE CodigoProducto='11679';

-- Insertas una nueva linea de detalles pedido con ese producto.

INSERT INTO DetallePedidos (CodigoPedido, CodigoProducto, Cantidad, PrecioUnidad, NumeroLinea)

VALUES(1, '11679', 2, 7, 6);

-- Compruebas que hay 2 menos en stock

SELECT CantidadEnStock FROM Productos WHERE CodigoProducto='11679';

3. TRIGGER PARA REGISTRAR CAMBIOS DE ESTADO EN PEDIDOS

Crear un trigger que registre en una tabla llamada historial_estado_pedido cada vez que se actualice el estado (estado) de un pedido en la tabla pedidos. Esta tabla debe almacenar el codigoPedido, el estadoAnterior, el estadoNuevo, y la fechaCambioPie de foto.

CREATE TABLE historial_estado_pedido (

```
  id INT AUTO_INCREMENT PRIMARY KEY,
  codigoPedido INT,
  estadoAnterior VARCHAR(20),
  estadoNuevo VARCHAR(20),
  fechaCambio DATETIME
);
```

3.1 Para probar el Trigger

```
-- Asegúrate de tener un pedido en la tabla pedidos:

SELECT codigoPedido, estado FROM Pedidos LIMIT 1;

-- Actualiza el estado de ese pedido:
UPDATE Pedidos
SET estado = 'Devuelto'
WHERE codigoPedido = 1;


-- Comprueba que se haya registrado el cambio:

SELECT * FROM historial_estado_pedido WHERE codigoPedido = 1;
```

4. TRIGGER PARA EVITAR PEDIDOS CON CANTIDADES NEGATIVAS

Crear un trigger que impida insertar líneas en la tabla `detalle_pedido` si la cantidad de productos es menor o igual a cero. Si se intenta, debe lanzar un error.

4.1 Para probar el Trigger

```
-- Intenta insertar una línea con cantidad negativa:
INSERT INTO DetallePedidos (codigoPedido, codigoProducto, cantidad, precioUnidad, numeroLinea)
VALUES (1, 'FR-123', -5, 10.5, 1);
```

-- Verás el siguiente error:

ERROR 1644 (45000): La cantidad del producto debe ser mayor que cero.

--Intenta con un valor válido para comprobar que sí se permite:

INSERT INTO DetallePedidos (codigoPedido, codigoProducto, cantidad, precioUnidad, numeroLinea)

VALUES (1, 'FR-105', 5, 10.5, 2);

5. CURSOR PARA MOSTRAR CLIENTES DE UNA CIUDAD

Crear un procedimiento almacenado que reciba como parámetro el nombre de una ciudad y recorra, usando un cursor, todos los clientes de esa ciudad mostrando su nombre y su teléfono mediante SELECT.

5.1 Cómo probar el cursor

-- llamar al procedimiento con una ciudad real (por ejemplo "Madrid"):

CALL mostrar_clientes_ciudad('Madrid');

El resultado serán varias consultas SELECT que devuelven los nombres y teléfonos de los clientes que viven en esa ciudad.

6. EJERCICIO 5: CURSOR PARA CALCULAR EL TOTAL DE COMPRAS DE CADA CLIENTE.

Crear un procedimiento almacenado que recorra todos los clientes y calcule el total gastado por cada uno en todos sus pedidos. Mostrar el nombre del cliente junto con el total gastado.

Pista: *Debes usar las tablas `Clientes`, `Pedidos` y `DetallePedidos`.*

Se usa `IFNULL` para devolver 0 si el cliente no ha hecho pedidos aún.

La acumulación se hace con una consulta individual dentro del bucle.

6.1 Cómo probar el cursor

-- Llamada

CALL total_por_cliente();

Verás un conjunto de resultados con el nombre de cada cliente y el total gastado en todos sus pedidos.

7. EJERCICIO 6: CURSOR PARA LISTAR PRODUCTOS CON BAJO STOCK

Crear un procedimiento almacenado que recorra todos los productos usando un cursor y muestre aquellos cuya cantidad en stock sea inferior a 50 unidades, mostrando su nombre, código y cantidad disponible.

7.1 Cómo probar el cursor

-- Llama al procedimiento:

CALL productos_bajo_stock();

Obtendrás una serie de resultados con los productos que tienen bajo nivel de stock.

8. EJERCICIO 7: CURSOR PARA CONTAR EMPLEADOS POR OFICINA.

Crear un procedimiento almacenado que recorra todas las oficinas usando un cursor y muestre el nombre de la ciudad y la cantidad de empleados que trabajan en cada una de las oficinas, indicando la ciudad.

8.1 Cómo probar el cursor

CALL contar_empleados_por_oficina();

Verás un conjunto de resultados con oficinas, ciudades y número de empleados por oficina indicando de qué ciudad son.

9. EJERCICIO 8: CURSOR PARA CONTAR EMPLEADOS POR CIUDAD.

Crear un procedimiento almacenado que recorra todas las oficinas usando un cursor y muestre el nombre de la ciudad y la cantidad de empleados que trabajan en cada una DE LAS CIUDADES.

9.1 Cómo probar el cursor

```
CALL emplePorCiudad();


-- Verás un conjunto de resultados con ciudades y número de empleados por ciudad.
```

10. EJERCICIO 9: FUNCIÓN PARA CALCULAR EL TOTAL DE UN PEDIDO.

Crear una función que reciba el `codigoPedido` como parámetro y devuelva el total del importe de ese pedido (cantidad × precioUnidad de cada línea).

10.1 Cómo probar la función

```
-- Llama a la función con el ID de un pedido existente:
SELECT total_pedido(1);


-- Para ver qué contiene ese pedido:
SELECT * FROM detalle_pedido WHERE codigoPedido = 1;
```